

PostFlow AI

Product Requirements Document

LinkedIn AI Post Generation & Team Publishing Platform

Version	2.0 — Tech Stack Edition
Date	March 2026
Status	Draft — For Engineering Review
Product Name	PostFlow AI
Frontend	Next.js 14 (App Router) + TypeScript + Tailwind CSS
Backend	Next.js API Routes (Serverless) on Vercel
Database	Supabase (Managed PostgreSQL)
Auth	Clerk (email/password + Google + GitHub OAuth)
AI Engine	OpenAI GPT-4o API
Payments	Stripe (Checkout + Webhooks + Customer Portal)
Deployment	Vercel (frontend + serverless API)

Table of Contents

1.	Executive Summary	3
2.	Problem Statement & Market Opportunity	4
3.	Market Research & Competitive Gaps	5
4.	Target Audience & User Personas	7
5.	Product Vision & Positioning	9
6.	Core Workflow: Generate → Review → Approve → Calendar → Publish	10
7.	Feature Specifications	14
8.	Tech Stack Architecture (Next.js + Supabase + Clerk + Stripe)	18
9.	Database Schema	23
10.	API Specifications	27
11.	User Stories & Acceptance Criteria	32
12.	Pricing & Monetisation Strategy	35
13.	Go-to-Market Strategy	37
14.	Success Metrics & KPIs	39
15.	MVP Scope & Phased Roadmap	40
16.	Risk Analysis & Mitigation	43
17.	Team & Budget Requirements	44

1. Executive Summary

PostFlow AI is a SaaS platform that closes the single biggest gap in the LinkedIn content tool market: the complete absence of an integrated Generate → Review Queue → Approve → Calendar → Auto-Publish workflow for small-to-mid-sized B2B teams.

Built on a modern full-stack architecture — Next.js 14 App Router, Supabase PostgreSQL, Clerk authentication, OpenAI GPT-4o, and Stripe payments — PostFlow AI is designed for rapid MVP delivery and reliable scaling without a large engineering team.

Core Value Proposition

PostFlow AI lets a 3-to-15-person B2B team go from raw idea to published LinkedIn post — with AI generation, team review, one-click approval, and auto-scheduling — inside a single tool. No Notion, no Slack approvals, no manual copy-pasting between apps.

Key Highlights

- Frontend: Next.js 14 (App Router) with TypeScript and Tailwind CSS, deployed on Vercel
- Backend: Next.js API Routes (serverless functions) for all endpoints — no separate server
- Database: Supabase managed PostgreSQL with RLS policies and typed client
- Auth: Clerk for user management — email/password, Google OAuth, GitHub OAuth
- AI: OpenAI GPT-4o via REST API — structured JSON responses, per-user usage tracking
- Payments: Stripe Checkout + Customer Portal + Webhooks for subscription management
- Primary differentiator: Only LinkedIn-first tool combining AI generation WITH team approval workflow at SMB price point (\$29–149/mo)
- MVP timeline: 10–12 weeks; target Year 1 ARR ~\$504K from 500 paying workspaces

2. Problem Statement & Market Opportunity

2.1 The Core Problem

LinkedIn has become the dominant B2B growth channel for startups, SaaS companies, consultants, and agencies. Yet the process of consistently creating and publishing high-quality LinkedIn content remains deeply broken for teams. The problem compounds across three layers:

Layer 1 — Content Creation Is Time-Consuming

- Founders and marketers spend 3–5 hours per week writing LinkedIn posts from scratch
- AI tools exist but produce generic output that requires heavy editing to sound authentic
- No tool learns multiple individual voices simultaneously for a whole team

Layer 2 — Team Approval Is Chaotic

- No LinkedIn-native approval workflow exists in any affordable tool today
- Teams route drafts through Slack, email, or Notion — completely external to the publishing tool
- Approval chains involving a founder, marketing lead, and legal reviewer require 3+ back-and-forth rounds across multiple apps

Layer 3 — Publishing Is Disconnected from Strategy

- Content calendar in one tool; scheduling in another; performance data in a third
- LinkedIn company page organic reach dropped 60–66% in 2025 — coordinated employee personal profile posting (employee advocacy) is now required but no affordable tool manages this

2.2 Market Opportunity

Segment	Estimated Size	Willingness to Pay
LinkedIn users globally	1 billion+	—
Active B2B content creators	~50 million	—
B2B teams (3–15 people) on LinkedIn	~8 million workspaces	\$49–149/mo
Agencies managing client LinkedIn	~500K globally	\$149–299/mo
TAM (1% of B2B teams)	~80,000 workspaces	\$65–84 ARPU

3. Market Research & Competitive Gaps

3.1 Competitive Landscape

Tool	AI Gen	Approval	Calendar	Auto-Pub	Voice	Team Roles	Price/mo
Taplio	Yes	No	Yes	Yes	Partial	No	\$39–199
AuthoredUp	No	No	Yes	Yes	No	No	\$20–45
Postwise	Yes	No	Yes	Yes	No	No	\$37–97
Supergrow	Yes	Partial	Yes	Yes	Yes	Partial	\$19–99
FeedHive	Yes	Yes	Yes	Yes	No	Yes	\$19–99
Hootsuite	Partial	Yes	Yes	Yes	No	Yes	\$99–249
Planable	No	Yes	Yes	Yes	No	Yes	\$33–179
PostFlow AI	Yes	Yes	Yes	Yes	Yes	Yes	\$29–149

3.2 Critical Market Gaps

Gap 1 — No End-to-End Team Workflow (Biggest Opportunity)

AI-first tools (Taplio, Postwise) have zero approval workflow. Team tools (Planable, Hootsuite) have weak AI.

No tool under \$149/mo combines both for LinkedIn-focused B2B teams of 3–15 people.

Teams currently use 3+ apps to manage a single post from idea to publish.

Gap 2 — Voice Training Requires Constant Manual Effort

Even tools claiming voice learning require extensive editing. No tool ingests existing posts + writing samples to build a locked-in voice profile per team member at company scale.

Gap 3 — No SMB-Priced Team Plan (\$49–149/mo)

Enterprise tools have team workflows (Hootsuite \$249+/mo). Solo tools are cheap (\$19–39/mo).

The 3-to-15-person B2B team willing to pay \$79–149/mo is completely unserved.

Gap 4 — No Performance Feedback Loop

Every current tool publishes and forgets. None automatically learns from top-performing posts to improve the next AI generation cycle — a self-improving content engine gap.

4. Target Audience & User Personas

4.1 Primary ICP

Attribute	Description
Company Type	B2B SaaS, tech startup, consulting firm, professional services
Team Size	3 to 15 people, at least 1 dedicated marketing or content role
LinkedIn Usage	Founder or team posting semi-regularly; LinkedIn is primary lead-gen channel
Pain Point	Inconsistent posting, approval chaos, AI output sounds too generic
Budget	\$79–149/mo per workspace
Decision Maker	Founder, Head of Marketing, or Content Lead
Tech Comfort	Comfortable with SaaS tools — GitHub, Notion, Stripe, Linear
Geography	US, UK, India, Canada, Australia (English-first)

4.2 User Personas

Persona 1: Arjun — B2B SaaS Founder (Primary)

Role	34, Co-founder & CEO, 12-person B2B SaaS startup
Goal	Post 3–4x/week on LinkedIn for inbound leads and thought leadership
Current Setup	ChatGPT for drafts → Slack for approval → manual LinkedIn post
Frustrations	Posts sound AI-generic; approval takes 2 days; drops to 1 post/week
WTP	\$79–99/mo for workspace with 3–5 users

Persona 2: Priya — Marketing Manager (Secondary)

Role	29, Marketing Manager, solo marketing dept at B2B SaaS
Goal	Manage LinkedIn for founder, 2 senior leads, and company page
Current Setup	Notion drafts → email review → manual copy-paste to LinkedIn
Frustrations	Founders miss review deadlines; no shared calendar; version confusion
WTP	\$79–149/mo Growth/Pro tier (company budget decision)

Persona 3: Nikhil — LinkedIn Ghostwriter / Agency (Upsell)

Role	36, Freelance LinkedIn ghostwriter, 8 client profiles
Goal	Produce 20+ posts/week across multiple clients efficiently

Current Setup	Separate Notion per client, manual scheduling, Google Cal reminders
Frustrations	Wrong client content posted; no client approval portal; chaos at scale
WTP	\$149–199/mo Agency tier — charges clients for the tool

5. Product Vision & Positioning

5.1 Vision Statement

Vision

PostFlow AI becomes the operating system for LinkedIn content teams — the single place where content is born, reviewed, approved, scheduled, and analysed, built on modern full-stack infrastructure that ships fast.

5.2 Positioning Statement

For B2B marketing teams and founders who need to maintain a consistent, authentic LinkedIn presence without the chaos of juggling multiple tools, PostFlow AI is the only LinkedIn-first content platform that combines AI generation, team approval workflows, and auto-publishing — unlike Taplio (no approval) or Hootsuite (no LinkedIn-quality AI) — built on Next.js, Supabase, and Clerk so it deploys in days, not months.

5.3 Core Design Principles

1. Workflow-first: Every feature serves the Generate → Review → Approve → Calendar → Publish flow
2. Full-stack type safety: TypeScript end-to-end from Supabase schema to React components
3. Serverless by default: Vercel + Next.js API Routes — no server management overhead
4. Auth-gate everything: Clerk middleware protects all /app routes server-side
5. LinkedIn-specific: Not a multi-channel social tool — every feature optimised for LinkedIn

6. Core Workflow: Generate → Review → Approve → Calendar → Publish

The five-stage workflow is the product's backbone. Each stage maps to specific Next.js API routes, Supabase tables, and Clerk-protected UI routes described in detail in Sections 9 and 10.

Stage 1: GENERATE — AI Post Creation

The user triggers post generation from the `/app/generate` route. The AI generation engine calls OpenAI GPT-4o via `/api/generate` (Next.js API Route), passing the user's voice profile, selected post format, topic, and optional source content.

Technical flow

6. User fills the generate form in Next.js App Router page (`/app/generate`)
7. React state manages: author selection, post format, topic input, source content, tone, length
8. On submit: POST `/api/generate` — Clerk middleware validates JWT, maps to Supabase users row
9. API Route fetches `voice_profile` from Supabase `brand_profiles` table for selected author
10. Constructs structured OpenAI GPT-4o prompt including voice profile JSON + topic + format
11. GPT-4o returns structured JSON response (3 variations per format)
12. Variations stored in `posts` table (status: 'draft') — `post_id` returned to frontend
13. Frontend displays 3 variations; user selects one → status updated to 'in_review'

Voice Profile System

- Each team member has a `voice_profile` record in Supabase `brand_profiles` table
- Profile built by: analysing up to 20 pasted existing LinkedIn posts via OpenAI analysis endpoint
- 10-question writing-style questionnaire stored as JSONB in `brand_profiles.questionnaire_answers`
- Voice profile embedded as system prompt context in every `/api/generate` call for that author
- Profile improves over time: approved/rejected variants feed back via `/api/voice/feedback` endpoint

Stage 2: REVIEW QUEUE — Structured Draft Management

Every generated post lands in the Review Queue at `/app/queue` — a server-side rendered Kanban board (Next.js App Router + Supabase real-time subscriptions) backed by the `posts` table.

Technical implementation

- `/app/queue` uses Next.js Server Components to SSR the initial queue state from Supabase
- Supabase Realtime subscriptions push live status updates without page refresh
- Post status transitions: `draft` → `in_review` → `changes_requested` → `approved` → `scheduled` → `published`
- Inline editing: PUT `/api/posts/:id` updates post content and creates a `post_versions` record

- Comments: POST /api/posts/:id/comments — stored in post_comments table with user_id FK
- @mention notifications: Make/n8n webhook triggers SendGrid email via /api/notifications/send

Stage 3: APPROVE — One-Click Approval Flow

The approval flow is designed for speed. Target: approver receives notification, reads post, approves in under 30 seconds — without necessarily opening the app.

Technical implementation

- Status change to in_review triggers Supabase Database Webhook → Next.js /api/webhooks/status
- Webhook handler calls SendGrid API to send approval email with signed one-click URLs
- Signed URL pattern: /api/approve?token=[JWT_signed_post_id]&action=[approve|reject]
- GET /api/approve validates Clerk JWT in token, updates post status in Supabase, sends confirmation
- Bulk approval: POST /api/posts/bulk-approve — updates multiple post IDs in single Supabase transaction
- Configurable approval rules stored per workspace in workspace_settings JSONB column

Stage 4: CALENDAR — Visual Content Scheduling

Approved posts appear in the Content Calendar at /app/calendar — a React calendar component powered by server-fetched Supabase data with client-side drag-and-drop scheduling.

Technical implementation

- /app/calendar: Next.js Server Component fetches approved posts for month view on server
- Drag-and-drop: client component updates scheduled_at via PATCH /api/posts/:id/schedule
- Optimal time suggestions: GET /api/posts/optimal-times returns AI-suggested slots based on timezone
- Gap detection: Supabase query finds days without scheduled posts and highlights them client-side
- Bulk schedule: POST /api/posts/bulk-schedule distributes posts across Mon–Fri at optimal times

Stage 5: AUTO-PUBLISH — Automated LinkedIn Publishing

At scheduled time, a Vercel Cron Job (/api/cron/publish) runs every 15 minutes, queries Supabase for due posts, and calls the LinkedIn API to publish.

Technical implementation

- Vercel Cron: vercel.json schedules /api/cron/publish every 15 minutes
- Cron handler queries Supabase: SELECT posts WHERE status='approved' AND scheduled_at <= NOW()
- For each post: calls LinkedIn REST API with author's OAuth token (stored encrypted in Supabase)

- On success: UPDATE posts SET status='published', linkedin_post_id=[id], published_at=NOW()
- On failure: retry 3x with exponential backoff → status='publish_failed' → SendGrid alert to admin
- LinkedIn OAuth tokens refreshed automatically via /api/auth/linkedin/refresh before each call

7. Feature Specifications

7.1 Feature Priority Matrix

Feature	Description	Priority	Phase
AI Post Generator	3 variations from topic + voice profile via GPT-4o	P0 — Must Have	MVP
Voice Profile Builder	Analyse posts + questionnaire → Supabase brand_profiles	P0 — Must Have	MVP
Review Queue (Kanban)	SSR Kanban with Supabase Realtime updates	P0 — Must Have	MVP
Approval Flow	Signed email URLs + Supabase status webhooks	P0 — Must Have	MVP
Content Calendar	Next.js Server Component + drag-and-drop client	P0 — Must Have	MVP
LinkedIn Auto-Publish	Vercel Cron + LinkedIn API OAuth publishing	P0 — Must Have	MVP
Team Roles & Permissions	Clerk user roles + Supabase RLS policies	P0 — Must Have	MVP
Stripe Subscriptions	Stripe Checkout + Webhooks + Customer Portal	P0 — Must Have	MVP
Post Format Templates	8 LinkedIn formats with format-specific prompting	P1 — Should Have	MVP
Content Source Ingestion	URL/PDF → GPT-4o summary → post generation	P1 — Should Have	Phase 2
Analytics Dashboard	Supabase queries + PostHog event tracking	P1 — Should Have	Phase 2
Slack Approval Integration	Slack API webhook via /api/integrations/slack	P2 — Nice to Have	Phase 2
Client Portal (Agency)	Separate /portal route with limited Clerk roles	P2 — Nice to Have	Phase 3
White-label Mode	Custom domain + subdomain routing on Vercel	P3 — Future	Phase 3

7.2 AI Post Generator — Detailed Spec

Input parameters (form state in /app/generate)

- author_id: UUID — FK to users table; pulls voice_profile from brand_profiles
- post_format: enum — Story | Insight | List | Hot Take | How-To | Behind-the-Scenes | Poll | Engagement
- topic_text: string (max 500 chars) — free text topic or idea

- `source_content`: optional — URL (fetched server-side) | File upload (Supabase Storage) | Text paste
- `tone`: enum — professional | conversational | bold | inspirational | educational
- `length`: enum — short (<150 words) | medium (150–300) | long (300–500) | thread
- `hook_style`: enum — question | bold_statement | surprising_stat | personal_story | contrarian

OpenAI GPT-4o structured output schema (response format: `json_object`)

```
{
  "variations": [
    {
      "id": 1,
      "content": "Full LinkedIn post text...",
      "char_count": 287,
      "hook_type": "personal_story",
      "hashtags": ["#B2BSaaS", "#ProductLed", "#Startup"]
    },
    { ... variation 2 ... },
    { ... variation 3 ... }
  ],
  "voice_match_score": 0.87,
  "meta": { "tokens_used": 412, "model": "gpt-4o" }
}
```

7.3 Voice Profile Builder — Detailed Spec

brand_profiles table fields (Supabase)

- `tone_score`: integer 1–5 (formal to casual) — extracted by GPT-4o analysis of sample posts
- `avg_sentence_length`: enum short|medium|long — extracted from sample post analysis
- `emoji_usage`: enum none|minimal|moderate|heavy — extracted from sample post analysis
- `hook_style_preferences`: text[] — ranked array of preferred hook types
- `signature_phrases`: text[] — up to 10 characteristic phrases extracted from samples
- `avoid_phrases`: text[] — user-defined exclusion list (up to 20 items)
- `cta_style`: enum soft|direct|question — user-defined CTA preference
- `brand_voice_keywords`: text[] — up to 10 adjectives describing the voice
- `questionnaire_answers`: jsonb — structured answers from 10-question onboarding flow
- `sample_posts_raw`: text — raw pasted posts used for initial analysis

8. Tech Stack Architecture

8.1 Full Stack Overview

Layer	Technology	Version / Plan	Role in PostFlow AI
Frontend	Next.js	14 (App Router)	Pages, layouts, server components, client components
Language	TypeScript	5.x	Type safety across frontend + API routes + DB queries
Styling	Tailwind CSS	3.x	Utility-first styling; shadcn/ui component library
Backend	Next.js API Routes	Serverless on Vercel	All API endpoints — no separate Express/NestJS server
Database	Supabase	PostgreSQL 15 + JS SDK	Users, posts, profiles, approvals, usage tracking
Auth	Clerk	Next.js SDK v5	User management, OAuth, session middleware, JWTs
AI Engine	OpenAI API	GPT-4o (json_object)	Post generation, voice analysis, content summarisation
Publishing	LinkedIn API	REST v2 + OAuth 2.0	Authenticated publishing to personal/company profiles
Payments	Stripe	Checkout + Webhooks	Subscriptions, plan gating, billing portal
Cron Jobs	Vercel Cron	vercel.json schedule	Auto-publish every 15 min, weekly digest emails
Email	SendGrid	REST API via fetch	Approval notifications, alerts, weekly digest
File Storage	Supabase Storage	S3-compatible	Image and PDF uploads for post content
Analytics	PostHog	JS SDK	User behaviour, funnel tracking, feature usage events
Deployment	Vercel	Pro plan	Frontend + serverless API + cron + edge config

8.2 Next.js App Router Structure

Directory layout

```
postflow-ai/
├── app/
│   ├── (auth)/
│   │   ├── sign-in/page.tsx
│   │   └── sign-up/page.tsx
```

Next.js App Router
Auth group (Clerk)

```

├── (app)/                                # Protected app group
│   ├── layout.tsx                        # Auth-gated layout
│   ├── dashboard/page.tsx
│   ├── generate/page.tsx
│   ├── queue/page.tsx
│   ├── calendar/page.tsx
│   ├── settings/page.tsx
│   └── onboarding/page.tsx
├── api/                                  # API Routes (serverless)
│   ├── auth/
│   │   ├── sync/route.ts
│   │   └── linkedin/route.ts
│   ├── posts/
│   │   ├── route.ts                    # GET list, POST create
│   │   ├── [id]/
│   │   │   ├── route.ts                # GET, PUT, DELETE
│   │   │   ├── approve/route.ts
│   │   └── comments/route.ts
│   ├── generate/route.ts                # OpenAI generation
│   ├── voice/
│   │   ├── analyse/route.ts
│   │   └── feedback/route.ts
│   ├── cron/
│   │   ├── publish/route.ts            # Vercel Cron
│   │   └── digest/route.ts
│   ├── stripe/
│   │   └── webhook/route.ts
│   └── webhooks/
│       └── status/route.ts
├── components/                          # Shared React components
├── lib/
│   ├── supabase/
│   │   ├── client.ts                  # Browser client
│   │   └── server.ts                  # Server client (cookies)
│   ├── openai.ts
│   ├── stripe.ts
│   └── linkedin.ts
├── types/                               # Shared TypeScript types
│   └── database.ts                    # Supabase generated types
├── middleware.ts                        # Clerk auth middleware
└── vercel.json                          # Cron + deployment config

```

8.3 Authentication: Clerk Setup

middleware.ts — route protection

```

import { clerkMiddleware, createRouteMatcher } from '@clerk/nextjs/server';

const isPublicRoute = createRouteMatcher([
  '/', '/sign-in(.*)', '/sign-up(.*)',
  '/api/stripe/webhook', // Stripe needs unauthenticated access
  '/api/approve',        // Email approval links (token-validated)
]);

export default clerkMiddleware((auth, req) => {

```

```
if (!isPublicRoute(req)) auth().protect();
});

export const config = {
  matcher: ['/(?!.*\._next.*)', '/', '/(api|trpc)(.*)'],
};
```

API Route auth pattern

```
// app/api/posts/route.ts
import { auth } from '@clerk/nextjs/server';
import { createSupabaseServerClient } from '@lib/supabase/server';

export async function GET() {
  const { userId } = auth();
  if (!userId) return Response.json({ error: 'Unauthorized' }, { status: 401 });

  const supabase = createSupabaseServerClient();
  const { data: user } = await supabase
    .from('users').select().eq('clerk_user_id', userId).single();

  const { data: posts } = await supabase
    .from('posts').select('*').eq('workspace_id', user.workspace_id)
    .order('created_at', { ascending: false });

  return Response.json({ posts });
}
```

8.4 Supabase Client Setup

lib/supabase/server.ts — server-side client (for API routes + Server Components)

```
import { createServerClient } from '@supabase/ssr';
import { cookies } from 'next/headers';
import type { Database } from '@types/database';

export function createSupabaseServerClient() {
  const cookieStore = cookies();
  return createServerClient<Database>({
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!,
    { cookies: { get: (n) => cookieStore.get(n)?.value } }
  });
}
```

8.5 OpenAI Generation Pattern

app/api/generate/route.ts — core generation endpoint

```
import { auth } from '@clerk/nextjs/server';
```



```
import OpenAI from 'openai';

const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

export async function POST(req: Request) {
  const { userId } = auth();
  if (!userId) return Response.json({ error: 'Unauthorized' }, { status: 401 });

  const { author_id, post_format, topic_text, tone, length } = await req.json();

  // 1. Fetch voice profile from Supabase
  const supabase = createSupabaseServerClient();
  const { data: profile } = await supabase
    .from('brand_profiles').select('*').eq('user_id', author_id).single();

  // 2. Check usage limits
  const usage = await checkUsageLimits(userId, supabase);
  if (usage.exceeded) return Response.json({ error: 'Limit exceeded' }, { status:
429 });

  // 3. Build prompt and call GPT-4o
  const completion = await openai.chat.completions.create({
    model: 'gpt-4o',
    response_format: { type: 'json_object' },
    messages: [
      { role: 'system', content: buildSystemPrompt(profile) },
      { role: 'user', content: buildUserPrompt(post_format, topic_text, tone,
length) },
    ],
    max_tokens: 2000,
  });

  // 4. Parse response, save to Supabase, update usage
  const result = JSON.parse(completion.choices[0].message.content!);
  await savePostDraft(result, author_id, supabase);
  await incrementUsage(userId, supabase);

  return Response.json(result);
}
```

8.6 Vercel Cron — Auto-Publish

vercel.json — cron schedule

```
{
  "crons": [
    {
      "path": "/api/cron/publish",
      "schedule": "* /15 * * * *"
    },
    {
      "path": "/api/cron/digest",
      "schedule": "0 8 * * 1"
    }
  ]
}
```

```
]
}
```

app/api/cron/publish/route.ts

```
import { NextRequest } from 'next/server';

export async function GET(req: NextRequest) {
  // Validate Vercel cron secret
  const authHeader = req.headers.get('authorization');
  if (authHeader !== `Bearer ${process.env.CRON_SECRET}`) {
    return new Response('Unauthorized', { status: 401 });
  }

  const supabase = createSupabaseServerClient();
  const now = new Date().toISOString();

  const { data: posts } = await supabase.from('posts')
    .select('*', users(linkedin_oauth_token))
    .eq('status', 'approved')
    .lte('scheduled_at', now);

  for (const post of posts ?? []) {
    const result = await publishToLinkedIn(post);
    const newStatus = result.success ? 'published' : 'publish_failed';
    await supabase.from('posts').update({
      status: newStatus,
      linkedin_post_id: result.postId ?? null,
      published_at: result.success ? now : null,
    }).eq('id', post.id);
    if (!result.success) await sendFailureAlert(post);
  }

  return Response.json({ processed: posts?.length ?? 0 });
}
```

8.7 Stripe Integration

app/api/stripe/webhook/route.ts — subscription sync

```
import Stripe from 'stripe';
import { headers } from 'next/headers';

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!);

export async function POST(req: Request) {
  const body = await req.text();
  const sig = headers().get('stripe-signature')!;

  const event = stripe.webhooks.constructEvent(
    body, sig, process.env.STRIPE_WEBHOOK_SECRET!
  );
}
```

```
const supabase = createSupabaseServerClient();

switch (event.type) {
  case 'customer.subscription.created':
  case 'customer.subscription.updated': {
    const sub = event.data.object as Stripe.Subscription;
    const tier = getPlanTier(sub.items.data[0].price.id);
    await supabase.from('users')
      .update({ subscription_tier: tier })
      .eq('stripe_customer_id', sub.customer);
    break;
  }
  case 'customer.subscription.deleted': {
    const sub = event.data.object as Stripe.Subscription;
    await supabase.from('users')
      .update({ subscription_tier: 'free' })
      .eq('stripe_customer_id', sub.customer);
    break;
  }
}
return Response.json({ received: true });
}
```

9. Database Schema (Supabase PostgreSQL)

All tables use UUID primary keys, UTC timestamps, and Row Level Security (RLS) policies. Supabase generates TypeScript types via supabase gen types typescript.

9.1 users Table

```
CREATE TABLE users (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  clerk_user_id     VARCHAR(255) UNIQUE NOT NULL,
  email             VARCHAR(255) NOT NULL,
  workspace_id      UUID REFERENCES workspaces(id) ON DELETE SET NULL,
  role              VARCHAR(50) NOT NULL DEFAULT 'author',
                  -- 'admin' | 'editor' | 'reviewer' | 'author'
  subscription_tier VARCHAR(50) NOT NULL DEFAULT 'free',
                  -- 'free' | 'growth' | 'pro' | 'agency'
  stripe_customer_id VARCHAR(255),
  linkedin_oauth_token TEXT,          -- encrypted
  linkedin_refresh_token TEXT,        -- encrypted
  created_at        TIMESTAMPTZ DEFAULT NOW(),
  updated_at        TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_users_clerk_user_id ON users (clerk_user_id);
CREATE INDEX idx_users_workspace_id ON users (workspace_id);
```

9.2 workspaces Table

```
CREATE TABLE workspaces (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name              VARCHAR(255) NOT NULL,
  owner_id          UUID REFERENCES users(id) ON DELETE CASCADE,
  subscription_tier VARCHAR(50) NOT NULL DEFAULT 'free',
  stripe_customer_id VARCHAR(255),
  settings          JSONB DEFAULT '{}',
                  -- approval_mode, auto_approve_after_hours, etc.
  created_at        TIMESTAMPTZ DEFAULT NOW(),
  updated_at        TIMESTAMPTZ DEFAULT NOW()
);
```

9.3 brand_profiles Table

```
CREATE TABLE brand_profiles (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  product_name      VARCHAR(255) NOT NULL,
  product_description TEXT NOT NULL,
  primary_audience TEXT NOT NULL,
  tone_score        INTEGER CHECK (tone_score BETWEEN 1 AND 5),
  avg_sentence_length VARCHAR(20) DEFAULT 'medium',
  emoji_usage       VARCHAR(20) DEFAULT 'minimal',
```

```

hook_style_preferences TEXT[] DEFAULT '{}',
signature_phrases TEXT[] DEFAULT '{}',
avoid_phrases TEXT[] DEFAULT '{}',
cta_style VARCHAR(50) DEFAULT 'direct',
brand_voice_keywords TEXT[] DEFAULT '{}',
questionnaire_answers JSONB DEFAULT '{}',
sample_posts_raw TEXT,
website_url TEXT,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(user_id)
);
CREATE INDEX idx_brand_profiles_user_id ON brand_profiles (user_id);

```

9.4 posts Table

```

CREATE TABLE posts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
  author_id UUID NOT NULL REFERENCES users(id),
  content TEXT NOT NULL,
  format_type VARCHAR(50) NOT NULL,
  -- 'story'|'insight'|'list'|'hot_take'|'how_to'|etc.
  status VARCHAR(50) NOT NULL DEFAULT 'draft',
  -- 'draft'|'in_review'|'changes_requested'|
  -- 'approved'|'scheduled'|'published'|'publish_failed'
  scheduled_at TIMESTAMPTZ,
  published_at TIMESTAMPTZ,
  linkedin_post_id VARCHAR(255),
  char_count INTEGER,
  hashtags TEXT[] DEFAULT '{}',
  topic_text TEXT,
  performance_data JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_posts_workspace_id_status ON posts (workspace_id, status);
CREATE INDEX idx_posts_author_id ON posts (author_id);
CREATE INDEX idx_posts_scheduled_at ON posts (scheduled_at) WHERE status =
'approved';

```

9.5 post_versions Table

```

CREATE TABLE post_versions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
  content TEXT NOT NULL,
  edited_by UUID REFERENCES users(id),
  version_num INTEGER NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX idx_post_versions_post_id ON post_versions (post_id, version_num);

```

9.6 post_comments Table

```
CREATE TABLE post_comments (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  post_id     UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,  
  user_id     UUID NOT NULL REFERENCES users(id),  
  content     TEXT NOT NULL,  
  resolved    BOOLEAN DEFAULT FALSE,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);  
CREATE INDEX idx_post_comments_post_id ON post_comments (post_id);
```

9.7 usage_tracking Table

```
CREATE TABLE usage_tracking (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  action_type VARCHAR(50) NOT NULL, -- 'generation'  
  month_year  VARCHAR(7)  NOT NULL, -- 'YYYY-MM'  
  count       INTEGER      NOT NULL DEFAULT 1,  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ DEFAULT NOW(),  
  UNIQUE(user_id, action_type, month_year)  
);  
CREATE INDEX idx_usage_user_action_month  
  ON usage_tracking (user_id, action_type, month_year);
```

9.8 Supabase Row Level Security (RLS) Policies

```
-- Enable RLS on all tables  
ALTER TABLE users          ENABLE ROW LEVEL SECURITY;  
ALTER TABLE posts          ENABLE ROW LEVEL SECURITY;  
ALTER TABLE brand_profiles ENABLE ROW LEVEL SECURITY;  
  
-- Users can only read/update their own record  
CREATE POLICY "users_own_row" ON users  
  USING (clerk_user_id = auth.jwt()->'sub');  
  
-- Workspace members can read/write workspace posts  
CREATE POLICY "workspace_posts" ON posts  
  USING (workspace_id IN (  
    SELECT workspace_id FROM users  
    WHERE clerk_user_id = auth.jwt()->'sub'  
  ));  
  
-- Only admin/editor roles can approve posts  
CREATE POLICY "approve_posts" ON posts  
  FOR UPDATE USING (  
    EXISTS (  
      SELECT 1 FROM users WHERE clerk_user_id = auth.jwt()->'sub'  
      AND role IN ('admin', 'editor')  
    )  
  ) WITH CHECK (status IN ('approved', 'changes_requested'));
```


10. API Specifications

All endpoints are Next.js App Router API Routes (/app/api/*). All routes (except those listed as public) require a valid Clerk JWT validated via Clerk middleware. Clerk userId is mapped to the internal Supabase users row via clerk_user_id.

10.1 Auth & User Endpoints

Endpoint	Method	Auth	Description
/api/auth/sync	POST	Clerk JWT	Create or fetch Supabase user record after Clerk login
/api/auth/linkedin	GET	Clerk JWT	Initiate LinkedIn OAuth 2.0 flow; store tokens in Supabase
/api/auth/linkedin/callback	GET	Public	LinkedIn OAuth callback; exchange code for tokens
/api/auth/linkedin/refresh	POST	Clerk JWT	Refresh expired LinkedIn OAuth token
/api/user/profile	GET	Clerk JWT	Fetch user + brand_profile; used by settings page
/api/brand-profile	POST	Clerk JWT	Create or update brand_profiles record
/api/usage	GET	Clerk JWT	Get current month usage counts vs plan limits

10.2 Post Management Endpoints

Endpoint	Method	Auth	Description
/api/posts	GET	Clerk JWT	List workspace posts; supports ? status=&author_id=&page=
/api/posts	POST	Clerk JWT	Create post draft manually (without AI generation)
/api/posts/[id]	GET	Clerk JWT	Fetch single post with versions and comments
/api/posts/[id]	PUT	Clerk JWT	Update post content; creates post_versions record
/api/posts/[id]	DELETE	Admin only	Delete post and cascade delete versions/comments
/api/posts/[id]/approve	POST	Editor/Admin	Approve or reject post; triggers SendGrid notification
/api/posts/[id]/comments	GET	Clerk JWT	List comments for a post
/api/posts/[id]/comments	POST	Clerk JWT	Add comment; triggers @mention notification if present

Endpoint	Method	Auth	Description
/api/posts/[id]/schedule	PATCH	Editor/Admin	Set or update scheduled_at datetime
/api/posts/bulk-approve	POST	Editor/Admin	Approve multiple post IDs in a single transaction
/api/posts/bulk-schedule	POST	Editor/Admin	Distribute multiple posts across optimal time slots

10.3 AI & Voice Endpoints

Endpoint	Method	Auth	Description
/api/generate	POST	Clerk JWT	Core AI generation — calls GPT-4o, returns 3 variations
/api/voice/analyse	POST	Clerk JWT	Analyse pasted posts + questionnaire → update brand_profiles
/api/voice/feedback	POST	Clerk JWT	Record approved/rejected variant to improve voice model
/api/posts/optimal-times	GET	Clerk JWT	Return suggested publish times for author's timezone

10.4 Stripe Endpoints

Endpoint	Method	Auth	Description
/api/stripe/create-checkout	POST	Clerk JWT	Create Stripe Checkout session for plan upgrade
/api/stripe/portal	POST	Clerk JWT	Create Stripe Customer Portal session for billing management
/api/stripe/webhook	POST	Public (sig verified)	Handle subscription events; update subscription_tier in Supabase

10.5 Cron & Webhook Endpoints

Endpoint	Method	Auth	Description
/api/cron/publish	GET	CRON_SEC RET header	Auto-publish scheduled posts via LinkedIn API every 15 min

Endpoint	Method	Auth	Description
/api/cron/digest	GET	CRON_SECRET header	Send weekly content digest email every Monday 8am
/api/approve	GET	Signed token	One-click email approve/reject (no login required)
/api/webhooks/status	POST	Supabase webhook	Handles DB-triggered status change events for notifications

11. User Stories & Acceptance Criteria

11.1 Onboarding Stories

US-001: Account Creation

AS A new user, I WANT TO create an account SO THAT I can start generating LinkedIn posts.

Acceptance Criteria:

- User can sign up via email/password using Clerk's hosted sign-up page at /sign-up
- Google OAuth and GitHub OAuth buttons available on the sign-up page (Clerk social providers)
- Clerk enforces email verification before user can access /app routes
- After verification: /api/auth/sync creates a Supabase users record (clerk_user_id mapped)
- User is redirected to /app/onboarding for brand profile setup (3-step flow)
- Onboarding completion triggers POST /api/brand-profile to create brand_profiles record

US-002: LinkedIn Account Connection

AS A user, I WANT TO connect my LinkedIn account SO THAT posts can auto-publish.

Acceptance Criteria:

- Settings page shows 'Connect LinkedIn' button; clicking opens OAuth flow via /api/auth/linkedin
- After successful OAuth: access_token and refresh_token stored encrypted in Supabase users table
- Connected state shown with profile picture and name fetched from LinkedIn API
- User can disconnect; tokens deleted from Supabase
- Expired token detected before publish attempt; re-auth notification sent via SendGrid

11.2 Core Workflow Stories

US-010: Generate a Post

AS AN author, I WANT TO generate LinkedIn post variations from a topic SO THAT I get quality drafts without writing from scratch.

Acceptance Criteria:

- /app/generate page loads with author selector, format picker, topic input, tone + length options
- On submit: POST /api/generate — Clerk JWT validated, usage limits checked against usage_tracking
- If free tier limit exceeded: response 429 with upgrade modal triggered on frontend
- GPT-4o returns 3 variations within 15 seconds; loading state shown during generation
- Each variation displays char count; exceeding 3,000 chars shows warning badge
- User selects 1 variation → POST /api/posts creates draft in Supabase, status='draft'
- User can edit inline then click 'Send for Review' → status updated to 'in_review'

US-020: Review Queue

AS A reviewer, I WANT TO see all pending posts in one board
SO THAT I can review and comment without switching tools.

Acceptance Criteria:

- /app/queue renders server-side (Next.js Server Component) with initial data from Supabase
- Supabase Realtime subscription pushes status changes live to all workspace members
- Inline text editing sends PUT /api/posts/[id] which creates a post_versions record
- Comment submission sends POST /api/posts/[id]/comments; @mentions trigger SendGrid email
- Reviewer can drag card to 'Approved' column → PATCH updates status to 'approved' in Supabase
- Only users with role 'admin' or 'editor' can move cards to Approved (enforced by Supabase RLS)

US-030: Email Approval

AS A founder/approver, I WANT TO approve posts from email
SO THAT I don't need to log into the app for quick reviews.

Acceptance Criteria:

- Status change to 'in_review' triggers Supabase DB webhook → /api/webhooks/status
- Webhook calls SendGrid with email containing full post text and signed approve/reject URLs
- Signed URL format: /api/approve?token=[JWT_signed_payload]&action=approve|reject
- GET /api/approve: validates JWT signature, not expired (1 hour TTL), updates Supabase post status
- Reject action requires comment — URL opens reject page with comment textarea before finalising
- Approver receives SendGrid confirmation email with post details after action

US-040: Calendar Scheduling

AS AN editor, I WANT TO schedule approved posts on a visual calendar
SO THAT posts publish at optimal times automatically.

Acceptance Criteria:

- /app/calendar Server Component fetches current month's posts from Supabase
- Drag-and-drop reschedule calls PATCH /api/posts/[id]/schedule with new scheduled_at timestamp
- 'Smart schedule' button calls GET /api/posts/optimal-times and auto-distributes posts
- Calendar highlights days with no scheduled posts using Supabase query for gaps
- Time conflict (2 posts for same author at same time) shows warning; user must resolve

US-050: Auto-Publish

AS A team, I WANT posts to publish automatically at scheduled time
SO THAT no one needs to manually log in and post.

Acceptance Criteria:

- Vercel Cron triggers /api/cron/publish every 15 minutes (configured in vercel.json)
- Cron queries Supabase for posts WHERE status='approved' AND scheduled_at <= NOW()
- For each post: calls LinkedIn REST API with author's decrypted OAuth token

- On success: Supabase updated — status='published', linkedin_post_id set, published_at set
- On failure: retry 3x with exponential backoff; final failure sets status='publish_failed'
- Publish failure triggers SendGrid alert to workspace admin via /api/notifications/send

12. Pricing & Monetisation Strategy

12.1 Pricing Tiers

Feature	Starter \$29/mo	Growth \$79/mo	Pro \$149/mo	Agency Custom
User seats	1 (solo)	Up to 5	Up to 15	Up to 50+
LinkedIn profiles	1 personal	3 profiles	10 profiles	Unlimited
AI generations/mo	30	150	500	Custom
Voice profiles	1	5	15	Unlimited
Review queue	Yes	Yes	Yes	Yes
Approval workflow	Basic	Full team	Team + rules	Custom chains
Auto-publish	Yes	Yes	Yes	Yes
Team roles (Clerk)	No	Yes	Yes	Yes
Stripe billing	Self-serve	Self-serve	Self-serve	Custom invoice
Analytics	Basic	Standard	Advanced	Full + Export
Support	Community	Email	Email + Chat	Dedicated CSM

12.2 Stripe Implementation

- Stripe Checkout for initial plan selection — `/api/stripe/create-checkout` returns Checkout Session URL
- Stripe Customer Portal for plan changes and cancellation — `/api/stripe/portal` returns portal URL
- Stripe Webhooks at `/api/stripe/webhook` (unprotected; signature verified with `STRIPE_WEBHOOK_SECRET`)
- Webhook events handled: `customer.subscription.created`, `.updated`, `.deleted` → updates Supabase `users.subscription_tier`
- Plan-gating in `/api/generate`: check `users.subscription_tier` + `usage_tracking` count before calling GPT-4o
- Annual billing: 20% discount configured as Stripe coupon applied at Checkout
- 14-day free trial on Growth plan: configured via Stripe `price trial_period_days`

12.3 Year 1 Revenue Projections

Tier	Target Workspaces	Price/mo	Monthly Revenue	Annual Revenue
Starter (\$29)	150	\$29	\$4,350	\$52,200
Growth (\$79)	250	\$79	\$19,750	\$237,000

Tier	Target Workspaces	Price/mo	Monthly Revenue	Annual Revenue
Pro (\$149)	80	\$149	\$11,920	\$143,040
Agency (\$299 avg)	20	\$299	\$5,980	\$71,760
TOTAL	500 workspaces	~\$84 ARPU	~\$42,000/mo	~\$504,000 ARR

13. Go-to-Market Strategy

13.1 Three-Phase Launch Plan

Phase 1 — Pre-Launch (Weeks 1–6)

- Build waitlist landing page (Next.js static export on Vercel) — target 500+ signups before launch
- Post founder journey on LinkedIn daily — document building PostFlow AI with screenshots and workflow demos
- Direct outreach to 50 B2B founders and marketing managers — offer free beta access for feedback calls
- Prepare Product Hunt assets: tagline, description, demo GIF of full Generate → Publish workflow

Phase 2 — Launch (Weeks 7–10)

- Product Hunt launch — target Top 5 of the Day; coordinate with waitlist subscribers for upvotes
- AppSumo lifetime deal submission — 300–500 buyers in first 2 weeks for early ARR and testimonials
- Launch demo video: 90-second screen recording of full workflow — post on LinkedIn, embed on landing page
- Cold email sequence to B2B SaaS marketing managers via Apollo/Clay — focus on team approval pain point

Phase 3 — Growth (Months 3–12)

- SEO: target 'LinkedIn post scheduler for teams', 'AI LinkedIn approval workflow', 'LinkedIn content team tool'
- Partner with LinkedIn ghostwriters: free Agency tier for 30 days — ghostwriters become distribution channels
- Integration partners: Notion, Slack, HubSpot integrations built as Next.js API route handlers
- Content marketing: weekly LinkedIn post templates, hook formulas, engagement guides for SEO + trust

14. Success Metrics & KPIs

14.1 Product Health Metrics

Metric	Definition	MVP Target	6-Month Target
Activation Rate	% signups generating 1+ post in 7 days	50%	65%
Workflow Completion	% generated posts reaching Published status	40%	60%
Weekly Active Workspaces	Workspaces with 1+ post action in 7 days	60%	75%
Trial to Paid Conversion	% trial workspaces converting to paid	15%	22%
Monthly Churn	% paying workspaces cancelling per month	<8%	<4%
Posts Published via Tool	Total auto-published posts per month	1,000	10,000
Avg Approval Time	Queue entry to Approved status	<4 hrs	<2 hrs
NPS Score	Net Promoter Score, in-app survey	>30	>50

14.2 Business Metrics

- MRR: \$5,000 at month 2; \$20,000 at month 6; \$42,000 at month 12
- CAC: under \$150 blended; LTV:CAC ratio above 5:1 at steady state
- ARPU: \$84/mo blended; payback period under 6 months on fully-loaded CAC
- PostHog funnel: landing → signup → onboarding → first generation → paid conversion tracked at each step

15. MVP Scope & Phased Roadmap

15.1 MVP Sprint Plan (10–12 Weeks)

Week	Sprint Focus	Key Deliverables	Tech
1–2	Foundation	Next.js 14 App Router setup, TypeScript config, Tailwind, Clerk integration, Supabase schema + RLS, /api/auth/sync	Next.js, Clerk, Supabase
3–4	AI Generation	Voice profile builder, OpenAI GPT-4o integration via /api/generate, 3-variation output, brand_profiles table	OpenAI, Supabase
5–6	Review Queue	Kanban board with Server Components + Supabase Realtime, inline editing, post_versions, post_comments, status transitions	Next.js, Supabase
7	Approval Flow	SendGrid notification emails, signed JWT approval URLs via /api/approve, Supabase DB webhook → /api/webhooks/status	SendGrid, Supabase
8	Content Calendar	Server Component calendar view, drag-and-drop PATCH /api/posts/[id]/schedule, optimal time suggestions	Next.js, Supabase
9	Auto-Publish	LinkedIn OAuth 2.0 flow, encrypted token storage, Vercel Cron /api/cron/publish, failure alerts	LinkedIn API, Vercel
10	Team Roles	Clerk org-level roles mapped to Supabase user.role, RLS policy enforcement, workspace invitations	Clerk, Supabase
11	Stripe Payments	Stripe Checkout, Customer Portal, /api/stripe/webhook, plan gating in /api/generate, usage_tracking	Stripe
12	Launch Polish	Onboarding flow, empty states, error boundaries, PostHog analytics, responsive design, Product Hunt prep	All

15.2 Environment Variables Required

```
# .env.local
# Clerk
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk...
CLERK_SECRET_KEY=sk...
NEXT_PUBLIC_CLERK_SIGN_IN_URL=/sign-in
NEXT_PUBLIC_CLERK_SIGN_UP_URL=/sign-up

# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://xxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...
```

```
# OpenAI
OPENAI_API_KEY=sk-...

# Stripe
STRIPE_SECRET_KEY=sk_live...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live...
STRIPE_WEBHOOK_SECRET=whsec...
STRIPE_GROWTH_PRICE_ID=price...
STRIPE_PRO_PRICE_ID=price...

# LinkedIn
LINKEDIN_CLIENT_ID=...
LINKEDIN_CLIENT_SECRET=...
LINKEDIN_REDIRECT_URI=https://app.postflow.ai/api/auth/linkedin/callback

# SendGrid
SENDGRID_API_KEY=SG....
SENDGRID_FROM_EMAIL=notifications@postflow.ai

# Vercel Cron
CRON_SECRET=...

# App
NEXT_PUBLIC_APP_URL=https://app.postflow.ai
JWT_SIGNING_SECRET=... # For signed email approval URLs
```

15.3 Post-MVP Roadmap

Phase	Timeline	Features + Tech Notes
Phase 2	Months 4–6	Content source ingestion (URL via Firecrawl API, PDF via Supabase Storage + GPT-4o), analytics dashboard (Supabase queries + PostHog), Slack approval integration via /api/integrations/slack
Phase 3	Months 7–9	Agency client portal (/portal route with limited Clerk role), AI performance learning (top posts → brand_profiles feedback loop), post thread support, Canva API integration for image posts
Phase 4	Months 10–12	Mobile PWA (Next.js PWA config), HubSpot/Salesforce CRM integrations as Next.js API route webhooks, advanced analytics export, team activity audit log via Supabase audit table

16. Risk Analysis & Mitigation

Risk	Severity	Description	Mitigation
LinkedIn API access denied	High	LinkedIn MDP approval needed for auto-publish; 2–4 week wait	Apply early; use Phantombuster as interim; architecture is API-agnostic
LinkedIn ToS changes	High	Could restrict third-party publishing at any time	Build manual download fallback; monitor LinkedIn developer policies
OpenAI cost spikes	Medium	GPT-4o cost at scale could squeeze margins	Enforce per-tier token limits; cache identical prompts; monitor usage_tracking table
Vercel cold starts	Medium	Serverless cold starts on /api/generate could slow UX	Use Vercel Pro for warmed functions; show generous loading state; consider streaming
Supabase RLS bypass	Medium	Misconfigured RLS could expose cross-workspace data	Write integration tests for every RLS policy; use service_role key only in server-side code
Clerk pricing changes	Low	Clerk's free tier limits may be hit at scale	Budget for Clerk Pro (\$25/mo+); consider migrating auth to Supabase Auth at scale
Competitor feature parity	Medium	Taplio or FeedHive could add approval workflow	Build deep voice learning; focus on no-code-friendly integration ecosystem

17. Team & Budget Requirements

17.1 MVP Team

Role	Hours/Week	Responsibilities
Full-Stack Developer	40 hrs	Next.js 14 App Router, TypeScript, Supabase schema + RLS, Clerk integration, OpenAI API, Stripe webhooks, Vercel deployment, LinkedIn OAuth
UI/UX Designer	15 hrs	Design system (Tailwind + shadcn/ui), onboarding flow, Kanban board, calendar view, responsive layouts
Product Lead / Founder	30 hrs	User research, OpenAI prompt engineering, GTM, feedback loops, copywriting

17.2 Monthly Infrastructure Costs

Service	Plan	Est. Cost/mo	Notes
Vercel	Pro	\$20	Serverless + cron + edge config + analytics
Supabase	Pro	\$25	Database + Realtime + Storage + Auth (if migrated)
Clerk	Pro	\$25–50	Auth for 500+ MAU; scales with users
OpenAI API	Pay-as-you-go	\$50–300	Scales with generation volume; ~\$0.05 per post
SendGrid	Essentials	\$20	Email notifications + weekly digest
Stripe	% transaction fees	Variable	2.9% + \$0.30 per transaction
LinkedIn API	Free (MDP)	\$0	Official Marketing Developer Platform
PostHog	Free tier	\$0–50	Up to 1M events/mo free; scale to paid later
TOTAL	—	~\$140–465/mo	Scales with user growth; well within early revenue

Document Version Control

Version 2.0 — March 2026 — Tech stack updated to align with reference architecture:

Next.js 14 App Router + Clerk + Supabase + OpenAI GPT-4o + Stripe + Vercel.

Update this document after: (a) first 10 beta user feedback sessions,

(b) LinkedIn API access confirmed or denied, (c) end of each sprint.
All RLS policies must be integration-tested before any production deployment.

— End of Document — PostFlow AI PRD v2.0 — Confidential